



# Security Audit

INDEX (DeFi)

# Table of Contents

|                                   |    |
|-----------------------------------|----|
| Executive Summary                 | 4  |
| Project Context                   | 4  |
| Audit Scope                       | 7  |
| Security Rating                   | 8  |
| Intended Smart Contract Functions | 9  |
| Code Quality                      | 10 |
| Audit Resources                   | 10 |
| Dependencies                      | 10 |
| Severity Definitions              | 11 |
| Status Definitions                | 12 |
| Audit Findings                    | 13 |
| Centralisation                    | 35 |
| Conclusion                        | 36 |
| Our Methodology                   | 37 |
| Disclaimers                       | 39 |
| About Hashlock                    | 40 |

## CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

## Executive Summary

The INDX team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

## Project Context

Indxs is a DeFi-focused index platform that enables users to create, manage, and invest in custom crypto index products. Built on Ethereum, the protocol simplifies complex portfolio allocation by bundling multiple tokens into a single index token. Users and experts can launch curated strategies—ranging from DeFi assets to thematic baskets—and earn fees through tokenized index trading and staking mechanisms. This approach offers passive diversification, on-chain transparency, and the flexibility to monetize index creation while benefiting from yield opportunities.

**Project Name:** INDX

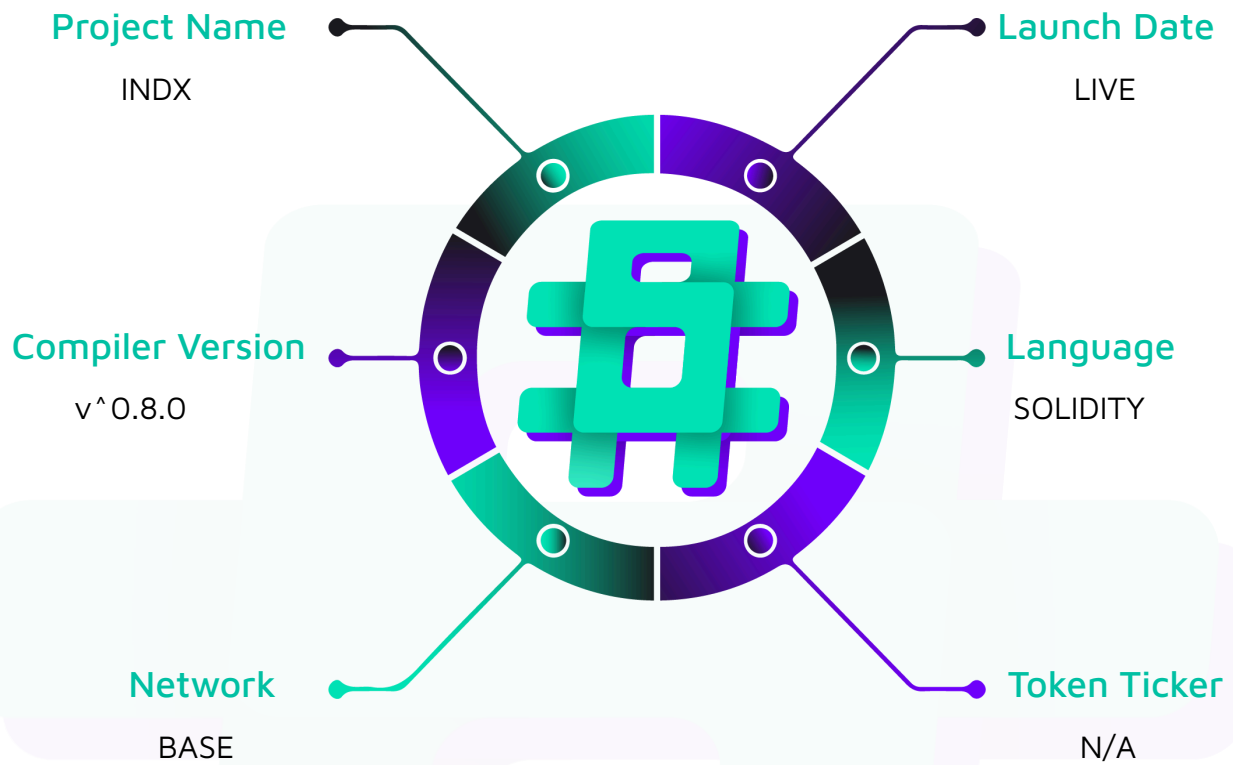
**Project Type:** DeFi

**Compiler Version:** ^0.8.0

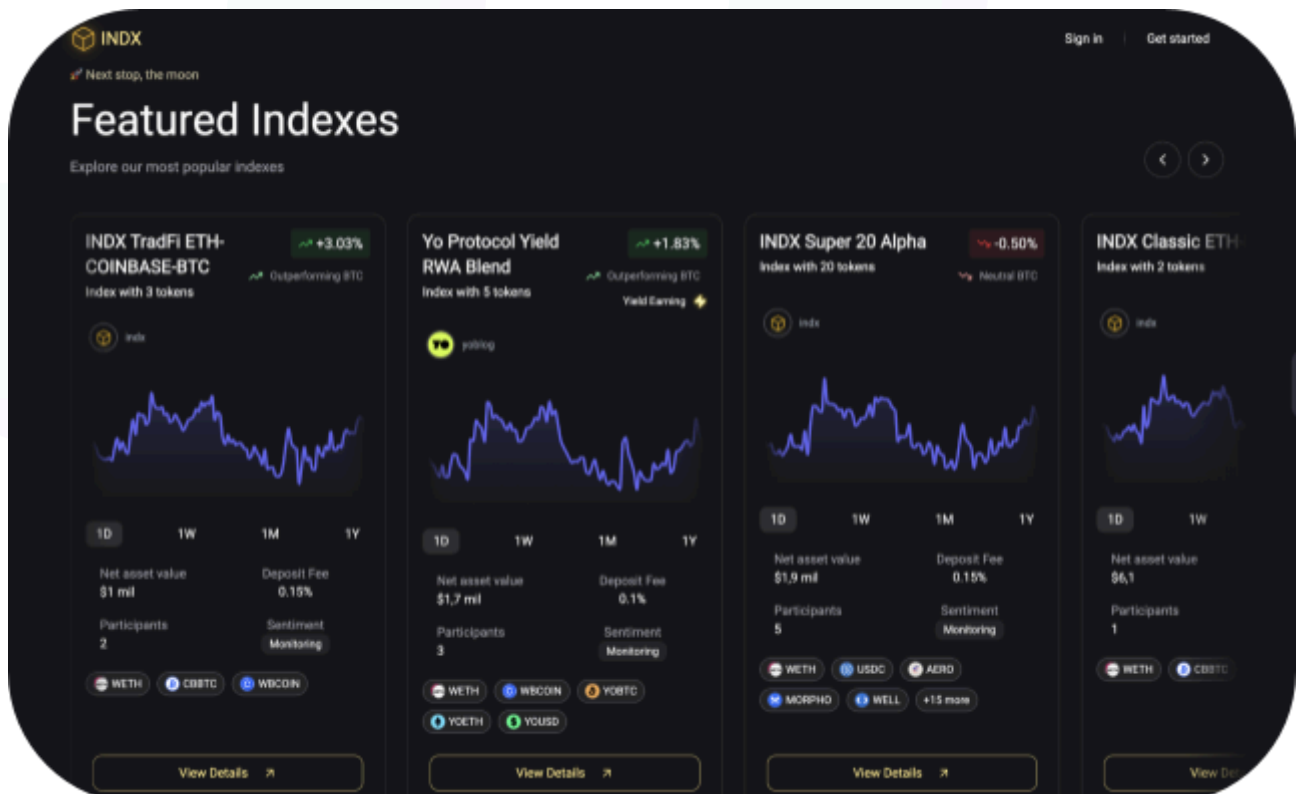
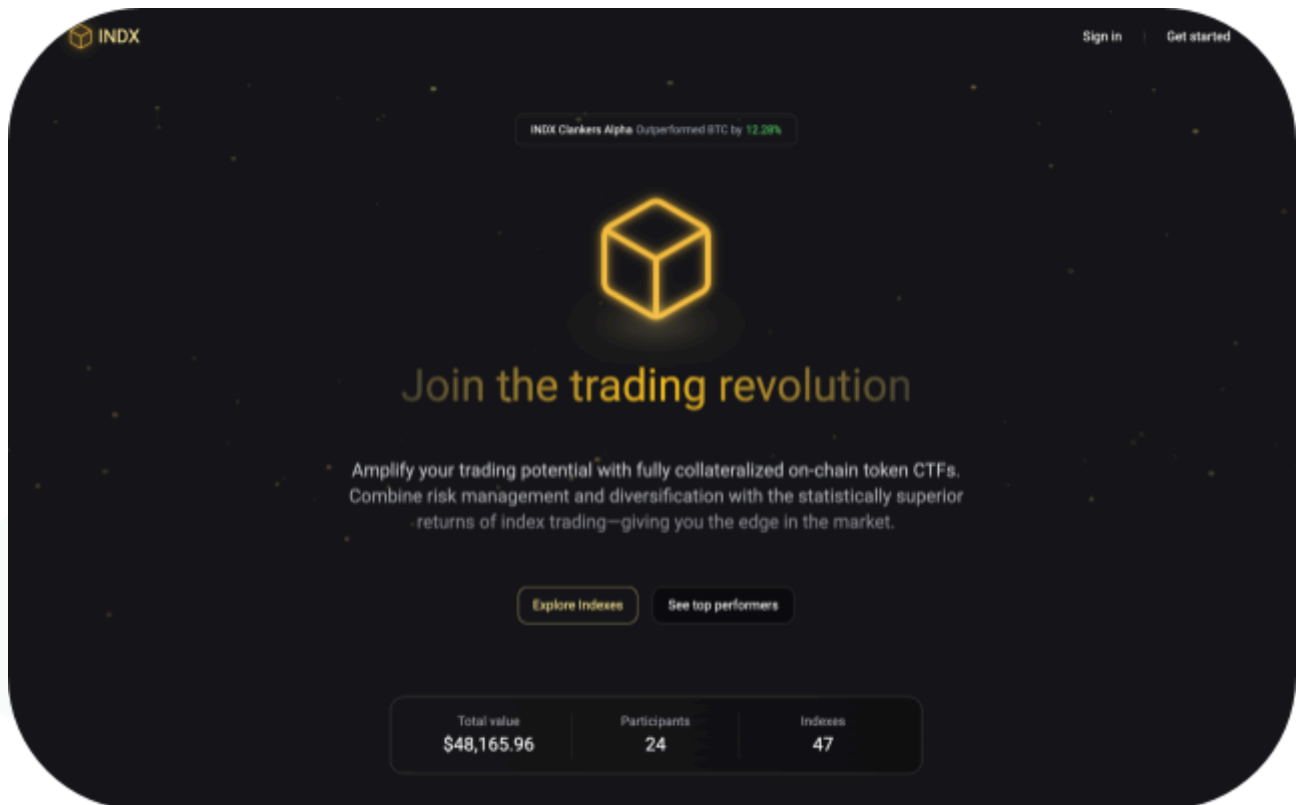
**Website:** <https://indxs.app/>

**Logo:**



**Visualised Context:**

## Project Visuals:



## Audit Scope

We at Hashlock audited the solidity code within the INDX project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

| Description         | INDX Smart Contracts             |        |                                          |
|---------------------|----------------------------------|--------|------------------------------------------|
| Platform            | Base / Solidity                  |        |                                          |
| Audit Date          | July, 2025                       |        |                                          |
| Contract 1          | index-v3.sol                     |        |                                          |
| Contract 1 MD5 Hash | 935b37d0ff68f00a020c6ab72dd967bc |        |                                          |
| Fix Commit Hash     | Review                           | GitHub | e6e8b9f60813801e2caab77df37d21f51177fd55 |
|                     |                                  |        |                                          |

## Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



*The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.*

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

### Hashlock found:

1 High severity vulnerability

3 Medium severity vulnerabilities

10 Low severity vulnerabilities

6 QA

**Caution:** Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

## Intended Smart Contract Functions

| Claimed Behaviour                                                                                                                                                                                                                                              | Actual Behaviour                             |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|
| <b>index-v3.sol</b> <ul style="list-style-type: none"><li>- Decentralized index fund manager</li><li>- Zap any token/ETH into the multi-token index</li><li>- Zap out or claim pro-rata shares</li><li>- Chainlink USD accounting and slippage guard</li></ul> | <b>Contract achieves this functionality.</b> |

## Code Quality

This audit scope involves the smart contracts of the INDX project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

## Audit Resources

We were given the INDX project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

## Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

## Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

| Significance | Description                                                                                                                                                                                           |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| High         | High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community. |
| Medium       | Medium-level difficulties should be solved before deployment, but won't result in loss of funds.                                                                                                      |
| Low          | Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.                                                                                        |
| Gas          | Gas Optimisations, issues, and inefficiencies.                                                                                                                                                        |
| QA           | Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.                            |

## Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

| Significance        | Description                                                                                                                                                                                                                                                                                                          |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Resolved</b>     | The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.                                                                                   |
| <b>Acknowledged</b> | The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception. |
| <b>Unresolved</b>   | The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.                                                                                                                                                                                               |

# Audit Findings

## High

**[H-01] index-v3#claimTokens** - ignoring refreshed USD valuations leads to value leakage on price increases

### Description

Because `claimTokens` calls `_calculateExpectedValues` but never uses its fresh USD estimate, it burns a user's old USD balance snapshot while transferring live token amounts at current prices. When prices rise, users extract more value than they paid.

### Vulnerability Details

```
function _processClaim(
    UserInfo storage user,
    address recipient,
    uint256 _percentage,
    IINDXAggregator.Trade[] memory _trades
) internal {
    uint256 price = getPriceFromChainlink();

    >>>    _calculateExpectedValues(user, _trades, price);    //@audit fresh (expectedEthAmt,
expectedUsdAmt) ignored

    // 1) transfer tokens at current balances

    uint256 claimAmount = (user.amounts[i] * _percentage) / FEE_DENOMINATOR;

    _transferToken(tokens[i], recipient, claimAmount);

    user.amounts[i] -= claimAmount;

    // 2) burn USD at the old snapshot value
```

```
uint256 claimedUsdAmount = (user.usdAmount * _percentage) / FEE_DENOMINATOR;  
  
user.usdAmount -= claimedUsdAmount;  
  
}
```

### Mathematical Example;

- User deposited 1 ETH when price = \$500 → user.usdAmount = 500.
- The price doubles to \$1000 after some time
- Claim 100%:
  - Transfers 1 ETH (worth \$1000 today)
  - Burns only 500 USD
  - Net gain: \$1000 received - \$500 burned = \$500

### Impact

Attackers or any user profiting from index appreciation can drain value from the remaining pool and other users.

### Recommendation

Use the fresh expected USD before burning by using `_calculateExpectedValues` returned values

### Status

Resolved

## Medium

**[M-01] index-v3#getPriceFromChainlink** -Missing L2 sequencer uptime validation exposes protocol to stale prices

### Description

The protocol operates on Base (an L2) but does not check the sequencer uptime feed before using Chainlink price data. During sequencer downtime, the last known price remains available but may be significantly stale, allowing attackers to exploit outdated prices when the sequencer comes back online.

### Vulnerability Details

The `getPriceFromChainlink` function directly queries the price feed without checking the sequencer status:

```
function getPriceFromChainlink() public view returns (uint256) { //@audit Should import
sequencer uptime feed and query first

    if (PRICE_FEED == address(0)) return 0;

    try AggregatorV3Interface(PRICE_FEED).latestRoundData() returns (
        uint80, int256 answer, uint256, uint256 updatedAt, uint80
    ) {

        if (block.timestamp - updatedAt > 1 days || answer <= 0) return 0;

        // ... uses price data without sequencer check

    }
}
```

When the L2 sequencer goes down, price feeds stop updating but remain accessible with stale data. When the sequencer restarts, there's a grace period where prices haven't been updated to current market values, but transactions can execute using outdated prices.

## Impact

Attackers can exploit the time window after the sequencer restarts to perform arbitrage or extract value using stale prices, potentially draining significant funds from the protocol through favorable swaps at outdated rates.

## Recommendation

Implement sequencer uptime validation before using price data:

```
// Add sequencer feed address

address constant SEQUENCER_UPTIME_FEED = 0xBCF85224fc0756B9Fa45aA7892530B47e10b6433;

function getPriceFromChainlink() public view returns (uint256) {

    // Check sequencer status first

    (, int256 answer, uint256 startedAt,,) =
    AggregatorV3Interface(SEQUENCER_UPTIME_FEED).latestRoundData();

    // 0 = sequencer up, 1 = sequencer down

    bool isSequencerUp = answer == 0;

    if (!isSequencerUp) return 0;

    // Make sure sequencer has been up for sufficient time

    if (block.timestamp - startedAt <= GRACE_PERIOD_TIME) return 0;

    // Then proceed with price feed query...

}
```

## Status

Resolved

## [M-02] index-v3#getPriceFromChainlink - Excessive staleness threshold allows outdated price usage

### Description

The price staleness check uses 1 day as the threshold, but the ETH/USD feed on Base has a 20-minute heartbeat. This means prices can be stale for up to 23+ hours and still be considered valid, exposing the protocol to significant price manipulation risks.

### Vulnerability Details

```
if (block.timestamp - updatedAt > 1 days || answer <= 0) return 0;
```

With a 20-minute heartbeat, prices should be rejected if they're more than a few heartbeat intervals old (e.g., 1-2 hours maximum). Using a 1-day threshold means the protocol will accept prices that are potentially 23+ hours stale during network issues or oracle outages.

### Impact

Stale prices can be exploited for significant arbitrage opportunities, allowing users to deposit or withdraw at favorable rates that don't reflect current market conditions. This can result in unfair value extraction from other users and protocol reserves.

### Recommendation

Use a staleness threshold appropriate for the feed's heartbeat:

```
uint256 constant STALENESS_THRESHOLD = 3600; // 1 hour for 20-minute heartbeat  
  
//...//  
  
if (block.timestamp - updatedAt > STALENESS_THRESHOLD || answer <= 0) return 0;
```

### Status

Resolved

## **[M-03] index-v3#processInactiveTokens** - A bad token transfer can DoS all user operations

### **Description**

Before every user action, `_processInactiveTokens` loops over removed tokens and does;

```
tokens[i].transfer(user, balance);
```

via `ResolvedZeppelin's safeTransfer`, which reverts on failure. If one inactive token becomes malicious (or misbehaving for any reason including the contract being blacklisted), this single revert blocks all operations (`zapIn`, `zapOut`, `claim`) for that user.

### **Recommendation**

Wrap each transfer in a `try/catch` (or low-level call) so that one bad token can't block processing of the others:

```
try IERC20(tokens[i]).transfer(user, balance) {  
    // ...  
} catch {  
    // log and continue  
}
```

### **Status**

Resolved

## Low

**[L-01] IINDXAggregator#Trade** - amountOut field is unused, leaving users vulnerable to unlimited slippage

### Description

The IINDXAggregator.Trade struct contains the amountOut field that is never used in the code, missing an opportunity to enforce user-specified minimum output amounts for slippage protection.

```
struct Trade {  
    uint256 amountIn;  
  
    uint256 amountOut; // @audit L this is never used in the code. huge slippage can go unnoticed  
  
    address[] path;  
  
    address[] adapters;  
}
```

Throughout the codebase, only amountIn, path, and adapters are used, while amountOut is completely ignored. This means users cannot specify minimum output expectations, potentially allowing trades to execute with significant slippage.

### Impact

Users have no protection against excessive slippage during swaps, potentially resulting in unfavorable trade execution and value loss.

### Recommendation

Either utilize the amountOut field for slippage protection or remove it from the struct to avoid confusion.

### Status

Resolved

## [L-02] index-v3#addToken - Duplicate WETH can be added

### Description

The `addToken` function can accidentally add WETH twice to the tokens array due to insufficient validation logic that confuses "unknown token" with "token at index 0".

In the `addToken` function:

```
function addToken(address _token) external onlyOwnerOrCurator {  
    if (_token == address(0)) revert InvalidAddress();  
    if (tokenIndex[_token] != 0 && isTokenActive[_token]) revert TokenAlreadyActive();  
    //@audit the owner can add WETH again.  
    // ...  
}
```

Since WETH is at index 0 and `tokenIndex[WETH] == 0`, the condition `tokenIndex[_token] != 0` evaluates to false for WETH, allowing it to bypass the duplicate check and be added again.

### Impact

This breaks the invariant that WETH should always be at index 0, potentially causing routing issues and incorrect fund allocation in the fallback mechanism.

### Recommendation

Add explicit check for WETH duplication:

```
if (_token == WNATIVE) revert TokenAlreadyActive();
```

### Status

Resolved

## [L-03] index-v3#convertToEth - Hardcoded minimum amount discriminates against low decimal tokens

### Description

The hardcoded minimum amount check of 1000 wei discriminates against tokens with different decimal places, potentially rejecting valid small-value transactions.

In `_convertToEth`:

```
if (_amount <= 1000) revert AmountTooSmall(); //@audit this is against low decimal tokens - like GUSD will be rejected if less than 10 gUSD while 10 USDC is OK
```

This check treats all tokens equally, regardless of decimals. For example, 1000 wei of GUSD (2 decimals) represents 10 GUSD, while 1000 wei of USDC (6 decimals) represents 0.001 USDC.

### Impact

Legitimate small transactions with low decimal tokens will be rejected, limiting protocol accessibility.

### Recommendation

Implement decimal-aware minimum amount validation.

### Status

Acknowledged

## [L-04] index-v3#convertToEth - Fee on transfer tokens causes swap failures

### Description

The contract assumes exact transfer amounts but doesn't account for tokens that charge fees on transfer, leading to potential swap failures.

In `_convertToEth`:

```
IERC20(_token).safeTransferFrom(msg.sender, address(this), _amount); // @audit tax tokens
will have less balance here when transferred.

// ...

_trade.amountIn = _amount; // @audit And this will revert due to amountIn should be the
balance due to tax tokens
```

The code assumes the contract receives exactly `_amount` tokens, but tax tokens will result in less than `_amount` being received, causing the subsequent swap to fail.

### Impact

Transactions with tax tokens will fail.

### Recommendation

Check the actual balance received after the transfer:

```
uint256 balanceBefore = IERC20(_token).balanceOf(address(this));

IERC20(_token).safeTransferFrom(msg.sender, address(this), _amount);

uint256 actualAmount = IERC20(_token).balanceOf(address(this)) - balanceBefore;

_trade.amountIn = actualAmount;
```

### Status

Acknowledged

## [L-05] index-v3#processFees - Smart contract wallets/contracts may cause transaction failures

### Description

The contract uses direct ETH transfers to fee wallets without checking if they can receive ETH, potentially causing transaction failures with smart contract wallets.

In `_processFees`:

```
if (INDEXWallet == address(0)) revert InvalidAddress();  
    payable(INDEXWallet).transfer(INDEXFeeAmount); //@audit better to call as these can be  
smart wallets.  
//...//  
if (curatorWallet == address(0)) revert InvalidAddress();  
    payable(curatorWallet).transfer(curatorFeeAmount);
```

The transfer function only provides 2300 gas and will fail if the receiving address is a smart contract that requires more gas for its fallback function.

### Impact

Fee collection may fail if fee wallets are smart contracts, potentially blocking all `zapIn` operations and causing a denial of service.

### Recommendation

Use ``call`` instead of `transfer` for better compatibility:

### Status

Resolved

## [L-06] index-v3#estimateEthValue - Fallback price estimation creates massive accounting errors for high-value tokens

### Description

The `_estimateEthValue` function uses an arbitrary `/100` fallback when price estimation fails, creating 99% accounting errors for high-value tokens.

In `_estimateEthValue`:

```
function _estimateEthValue(address _token, uint256 _amount) internal view returns (uint256) {  
    // ...  
    } catch (bytes memory) {  
        return _amount / 100; // @audit    }  
}
```

For high-value tokens like WBTC, this fallback drastically undervalues the token (e.g., 1 WBTC worth \$100,000 becomes valued at \$1000), causing massive discrepancies in USD accounting during zapOut operations.

### Impact

User USD balances become wildly inaccurate, leading to poor UX and potential accounting issues in future contract upgrades.

### Recommendation

Implement a more sophisticated fallback mechanism or revert when price estimation fails for critical operations.

### Status

Resolved

## **[L-07] index-v3#addTokens** - Inactive tokens create a bottleneck by occupying token slots permanently

### **Description**

Inactive tokens remain in the tokens array permanently, potentially preventing new tokens from being added once the `MAX_TOKENS` limit is reached.

In `addTokens`:

```
if (tokens.length >= MAX_TOKENS) revert ExceedMaxTokens();
```

Once tokens are added and later deactivated, they continue to occupy slots in the tokens array, reducing the effective capacity for active tokens.

### **Impact**

Protocol flexibility is reduced as inactive tokens consume limited token slots, potentially preventing the addition of new valuable tokens.

### **Recommendation**

Implement a mechanism to remove inactive tokens from the array or increase `MAX_TOKENS` limit with proper gas optimization.

### **Status**

Acknowledged

## **[L-08] index-v3#zapIn** - Missing trade path validation allows incorrect swap routing

### **Description**

The `zapIn` function never validates that trade paths are properly structured, potentially leading to swaps with wrong token pairs.

```
function zapIn(  
    address recipient,  
    address _token,
```

```

uint256 _amount,

uint256[] memory _percents,

IINDXAggregator.Trade[] memory _trades,

uint256 _minTotalValueOut,

uint256 _slippage

) external payable onlyInitialized nonReentrant processInactiveTokens(recipient) {

    if (recipient == address(0)) revert InvalidAddress();

    if (_amount == 0) revert AmountTooSmall();

    if (_percents.length != NUM_TOKENS) revert InvalidPercents();

    if (_trades.length != NUM_TOKENS + 1) revert InvalidTradeConfig();

    if (_slippage > MAX_SLIPPAGE) revert SlippageTooHigh();

    uint256 totalPercentage = _validatePercentsForActiveTokens(_percents);

    uint256 ethAmount = _convertToEth(_token, _amount, _trades[0]);

    _processZap(recipient, _token, ethAmount, _percents, _trades, _minTotalValueOut,
_slippage, totalPercentage);

    _validateUserState(recipient);

}

```

Without validating that `path.length == adapters.length + 1`, and the path endpoints match expected tokens, users could accidentally execute swaps with completely unrelated tokens.

## Impact

Users may end up with unexpected tokens in their index allocation, breaking the intended composition and potentially losing value.

## Recommendation

Add path validation.

```
require(trade.path.length == trade.adapters.length + 1, "Invalid path length");  
require(trade.path[0] == expectedTokenIn, "Invalid path start");  
require(trade.path[trade.path.length - 1] == expectedTokenOut, "Invalid path end");
```

## Status

Acknowledged

## [L-09] index-v3#convertToEth & \_processFees - Inconsistent minimum amount validation across deposit flows

### Description

The contract applies minimum amount validation inconsistently across different deposit flows. Native ETH deposits bypass validation entirely, and the remaining ETH after fee deduction is not checked against minimum thresholds.

In `_convertToEth`:

```
function _convertToEth(address _token, uint256 _amount, IINDXAggregator.Trade memory
_trade) internal returns (uint256 ethAmount) {

    if (_token == address(0)) return msg.value; // @audit should check (_amount <= 1000)
    for all even after swap

    uint8 allowedMethod = factory.allowedTokens(_token);

    if (allowedMethod == 0) revert UnsupportedToken();

    if (_amount <= 1000) revert AmountTooSmall(); // Only applied to ERC20 tokens

    // ...

}
```

And in `_processFees`:

```
feeAmount = INDXFeeAmount + curatorFeeAmount;

if (_ethAmount <= feeAmount) revert AmountTooSmall();

totalEarned += (curatorFeeAmount * price) / 1 ether;

remainingEth = _ethAmount - feeAmount; // @audit for consistency it should check > 1000
wei
```

Native ETH deposits completely bypass the `AmountTooSmall` validation, and the remaining ETH after fee deduction is not validated against minimum thresholds, creating inconsistent behavior across deposit methods.

## Impact

Users can deposit tiny amounts of ETH or end up with very small remaining amounts after fees that may not be economically viable for swapping, leading to failed transactions or inefficient gas usage during token purchases.

## Recommendation

Apply consistent minimum amount validation across all flows:

```
// In _convertToEth

if (_token == address(0)) {

    if (msg.value <= 1000) revert AmountTooSmall();

    return msg.value;

}

// In _processFees

remainingEth = _ethAmount - feeAmount;

if (remainingEth <= 1000) revert AmountTooSmall();
```

## Status

Acknowledged

## [L-10] index-v3#\_addTokenEstimatedValue - Users can inject their own adapters for controlled returns

### Description

During a claim, the contract walks through the user's `_trades` array, fully controlled by the caller, and calls each adapter's `query()` to estimate ETH value. The user can inject their malicious adapter, and this can return an arbitrarily large `queryResult`.

```
for (uint256 j = 0; j < trade.adapters.length; j++) {
    try IAdapter(trade.adapters[j]).query(
        amount,
        trade.path[j],
        trade.path[j + 1]
    ) returns (uint256 queryResult) {
        amount = queryResult; // @audit ← attacker can set this to any value
    } catch { ... }
}
return ethAmount + amount;
```

Because `trade.adapters` is user-supplied, an attacker substitutes a malicious adapter that returns huge values.

### Impact

Although this inflates the local estimate in `_calculateExpectedValues`, those numbers are never used to move tokens or adjust on-chain balances. But this creates an uncontrolled input vector, possibly for next versions of the contract.

### Recommendation

Either remove this unused estimation path in `claimTokens`, or restrict adapters to a trusted whitelist. If future logic consumes these estimates for slippage or minimum value checks, ensure adapters come only from the protocol's approved list.

**Status**

Resolved

## QA

### **[Q-01]** `index-v3#initialize` - Swap aggregator address cannot be updated after initialization

#### **Description**

The swap aggregator address is set only during contract initialization, with no setter function available, limiting contract flexibility if the aggregator needs to be upgraded or changed.

#### **Recommendation**

Add an `updateAggregator` function restricted to the owner.

#### **Status**

Resolved

### **[Q-02]** `index-v3#transferCurator` - Curator role transfer should require owner approval for security

#### **Description**

The `transferCurator` function allows curators to transfer their role to any address without owner approval, which could lead to unauthorized role changes if the curator's address is compromised. As such, a compromised curator could transfer the role to an attacker-controlled address multiple times.

#### **Recommendation**

Require owner approval for curator transfers.

#### **Status**

Acknowledged

**[Q-03] index-v3#zapIn** - ETH sent with ERC20 deposits gets permanently locked

### Description

When users call `zapIn` with an ERC20 token but accidentally send ETH via `msg.value`, that ETH gets locked in the contract with no recovery mechanism. The function accepts both `msg.value` and ERC20 tokens simultaneously, but only processes the intended token type.

### Recommendation

Add validation to revert if `msg.value > 0` when `_token != address(0)`.

### Status

Acknowledged

**[Q-04] index-v3#safeSwapEth** - Hardcoded fee parameter may cause failures with different adapters or if the adapter changes its fee settings

### Description

The swap function uses a hardcoded fee of 0, which may not be compatible with all adapters if they change their fee structure. If adapters require specific fee values, the hardcoded 0 could cause swaps to fail.

### Recommendation

Parse the required fee from the adapter or make it configurable.

### Status

Acknowledged

**[Q-05] index-v3#convertToEth** - Missing `try/catch` for wrapper withdrawals, unlike other external calls

### Description

The wrapper withdrawal call lacks `try/catch` protection, while other external calls in the function are properly wrapped for error handling. While the aggregator swap calls use `try/catch` blocks, the wrapper withdrawal is called directly without error handling.

### Recommendation

Add `try/catch` protection for consistency

### Status

Acknowledged

**[Q-06] index-v3#MAX\_SLIPPAGE** - `MAX_SLIPPAGE` should be configurable rather than hardcoded constant

### Description

The maximum slippage is hardcoded at 5% which may not be suitable for all market conditions or trading scenarios. A fixed 5% slippage limit may be too restrictive during volatile market conditions or too permissive for stable markets.

### Recommendation

Make `MAX_SLIPPAGE` configurable per index or allow the owner/curator to adjust it.

### Status

Acknowledged

## Centralisation

The INDX project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

## Conclusion

After Hashlock's analysis, the INDX project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

# Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

## **Manual Code Review:**

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

## **Vulnerability Analysis:**

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

**Documenting Results:**

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

**Suggested Solutions:**

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

# Disclaimers

## Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

## Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

## About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

**Website:** [hashlock.com.au](https://hashlock.com.au)

**Contact:** [info@hashlock.com.au](mailto:info@hashlock.com.au)



**#hashlock.**